

RTL and Testbench Agent

ECEN 689 - LLMs for Agentic Hardware Design and Security

Nithin Gowtham Saravanan

UIN: 337001954

Assignment 1 Technical Report

LLM-Based RTL Generation, Self-Checking Testbench Generation, and Iterative RTL Repair

24 experimental runs • 4 hardware blocks • 3 prompt styles • 2 runs per style

Model: gpt-5.4-2026-03-05 • Provider: tamu • Maximum output: 4096 tokens

Fall 2026

Executive Summary

The report evaluates an LLM-based hardware-design workflow that generates SystemVerilog RTL, generates a self-checking testbench, validates the RTL using both the generated testbench and the instructor-provided sample tests, and invokes a bounded repair loop when the authoritative tests fail. The experiment covers ALU, RegFile, Extend, and Controller blocks using Direct, Rules & Constraints, and Design Planning prompt styles, with two runs for every block/style combination.

The study contains **24 runs, 7 repair calls, and 202,372 total recorded tokens**. The initial RTL achieved a mean instructor-suite score of **98.75%**, with **19/24** runs passing the instructor suite before repair. Rules & Constraints had the highest observed initial mean at **100.00%**.

The central verification finding is that generated-testbench success does not imply correctness. Some RTL versions passed their LLM-generated tests but failed instructor vectors, while RegFile RTLs passed the instructor suite even when their generated testbench failed. The instructor sample suite was therefore treated as the authoritative acceptance criterion, while generated testbenches were retained as a secondary diagnostic artifact.

The repair loop recovered **4/5** repaired runs. Four Controller runs reached 40/40 after one repair, while the ALU Design Planning run exhausted all three repair attempts and ended with a compile failure. This demonstrates both the value of evidence-driven repair and the need for bounded convergence criteria.

Engineering takeaway: *Reliable agentic RTL generation is not simply a prompting problem. The strongest architecture is specification-driven generation followed by independent verification, evidence-based bounded repair, and complete artifact logging.*

1. Introduction and Objectives

The assignment investigates an agentic hardware-design workflow in which an LLM is integrated with conventional EDA tooling. Instead of stopping after code generation, the workflow asks the model to produce RTL and verification collateral, executes the resulting design, interprets concrete failures, and revises the RTL when necessary. The implementation is generic across the four supplied hardware specifications rather than embedding hardware-specific repair logic.

The experiment has five objectives: (1) compare multiple prompt styles under a fixed model/configuration; (2) measure initial RTL quality before repair and preserve all versions; (3) use the instructor-provided sample tests as the authoritative correctness signal; (4) generate self-checking testbenches and evaluate their usefulness; and (5) preserve prompts, responses, metadata, token usage, RTL, testbenches, and simulation logs for reproducibility.

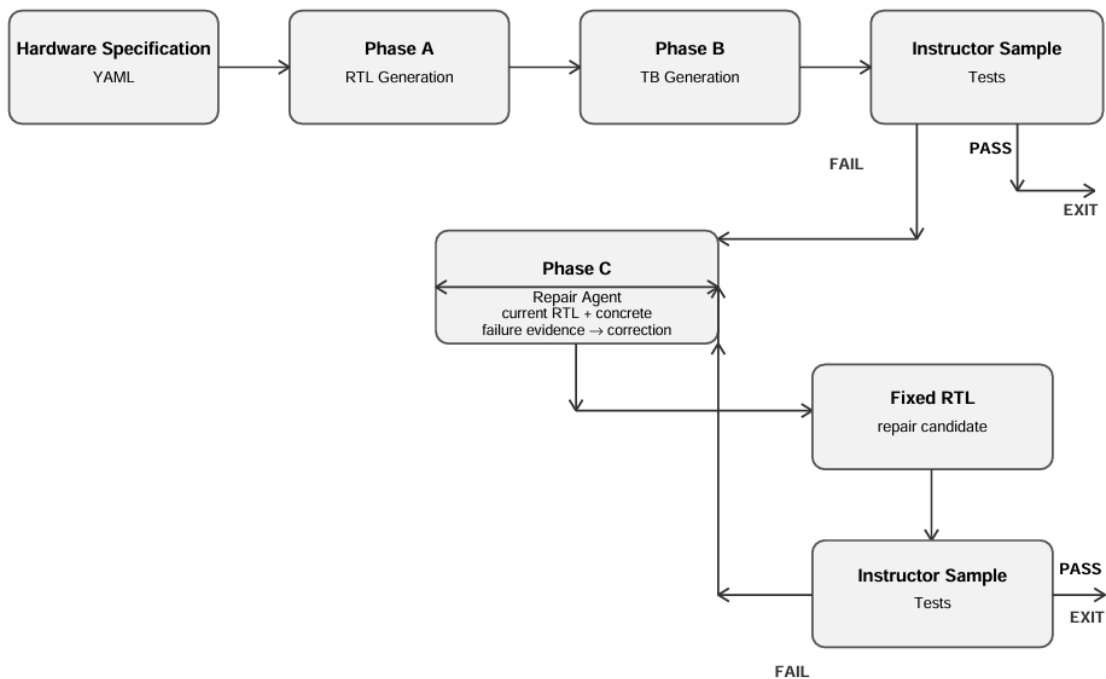
1.1 Hardware Blocks

Block	Function	Instructor tests
ALU	Arithmetic/logic datapath	40
RegFile	Register-file storage and read/write behavior	39
Extend	Immediate extension behavior	40
Controller	Instruction decode/control behavior	40

Table 1. Hardware blocks and authoritative instructor-test suite sizes.

2. Agent Architecture and Workflow

The workflow is divided into three controlled phases. Phase A translates the YAML specification into initial RTL. Phase B asks the model to produce a self-checking testbench. Phase C compiles and executes the generated testbench, runs the instructor sample suite, and conditionally enters a repair loop. The exact prompts and model metadata are stored for every LLM call.



Retry loop: maximum 3 repair attempts; stop immediately when instructor suite passes

Figure 1. High-level RTL/testbench agent architecture.

Phase A — RTL generation. The YAML hardware specification and selected prompt template are sent to the fixed LLM. The returned SystemVerilog is cleaned and stored as `initial_rtl.sv`, together with the exact prompt, response, model metadata, and token counts.

Phase B — testbench generation. The same hardware specification is provided to the LLM with a verification-oriented system prompt. The generated self-checking testbench is stored as `<top_module>_tb.sv` and reused for all Phase C RTL attempts for that run.

Phase C — simulation and repair. Icarus Verilog compiles and executes the generated testbench, then the instructor sample-test script is run against the same RTL. Only the instructor script result determines acceptance. On FAIL, the current RTL and concrete failure evidence are sent to the repair agent; the repair budget is three attempts, and the loop stops early after a pass.

Phase C decision rule: The generated testbench is not the acceptance oracle. A run stops successfully when the instructor sample script reports PASS.

3. Experimental Methodology

3.1 Controlled Variables

All runs used the same recorded model identifier, **gpt-5.4-2026-03-05**, the same provider (**tamu**), and the same maximum output setting of **4096 tokens**. Phase A prompt-style variation was isolated to the user prompt template while the Phase A system role remained fixed. Phase B and Phase C each used fixed role-specific system prompts. This design limits confounding from model/settings changes.

3.2 Prompt Styles

Direct: concise implementation instruction emphasizing exact adherence to the hardware specification.

Rules & Constraints: adds explicit requirements for interface preservation, operations, widths, signedness, logic type, and Icarus compatibility.

Design Planning: asks the model to identify interfaces, operations, corner cases, and combinational/sequential structure before producing RTL.

Each of the four blocks was run twice with each of the three prompt styles, producing 24 runs. The same generated testbench was used for all RTL attempts within a run, while repaired RTLs were saved separately.

3.3 Correctness and Compilation

The generated testbench was compiled and executed with Icarus Verilog. The instructor-provided sample-test script was then executed directly against each RTL version. Icarus messages such as "sorry: constant selects in always_* processes are not fully supported" were treated as tool warnings when the simulation completed and produced a test result. The instructor suite determines final RTL acceptance.

3.4 Artifact Logging

Every LLM call records the exact system prompt, exact user prompt, full model response, model identifier, settings, and token usage. Phase C attempt 0 is the initial validation and therefore has zero LLM tokens; only actual repair calls contribute Phase C token usage. All generated and repaired RTL versions remain in their original run directories.

4. Task 1 — Prompt-Style Experiments

The experiment satisfies the required design of at least three prompt styles and at least two runs per style for every hardware block. The table below reports the initial RTL result for each run and includes the total number of repair attempts triggered by that run.

Block	Prompt style	Run	Compiled?	Generated TB	Instructor	Initial score	Repairs
ALU	Design Planning	1	PASS	20/20 PASS	36/40 FAIL	90.0%	3
ALU	Design Planning	2	PASS	15/20 FAIL	40/40 PASS	100.0%	0
ALU	Direct	1	PASS	15/19 FAIL	40/40 PASS	100.0%	0
ALU	Direct	2	PASS	17/20 FAIL	40/40 PASS	100.0%	0
ALU	Rules & Constraints	1	PASS	18/20 FAIL	40/40 PASS	100.0%	0
ALU	Rules & Constraints	2	PASS	18/20 FAIL	40/40 PASS	100.0%	0
Controller	Design Planning	1	PASS	18/22 FAIL	39/40 FAIL	97.5%	1
Controller	Design Planning	2	PASS	19/20 FAIL	39/40 FAIL	97.5%	1
Controller	Direct	1	PASS	22/22 PASS	37/40 FAIL	92.5%	1
Controller	Direct	2	PASS	15/20 FAIL	37/40 FAIL	92.5%	1
Controller	Rules & Constraints	1	PASS	17/22 FAIL	40/40 PASS	100.0%	0
Controller	Rules & Constraints	2	PASS	20/20 PASS	40/40 PASS	100.0%	0
Extend	Design Planning	1	PASS	20/20 PASS	40/40 PASS	100.0%	0
Extend	Design Planning	2	PASS	22/22 PASS	40/40 PASS	100.0%	0
Extend	Direct	1	PASS	20/20 PASS	40/40 PASS	100.0%	0
Extend	Direct	2	PASS	22/22 PASS	40/40 PASS	100.0%	0
Extend	Rules & Constraints	1	PASS	20/20 PASS	40/40 PASS	100.0%	0
Extend	Rules & Constraints	2	PASS	22/22 PASS	40/40 PASS	100.0%	0
RegFile	Design Planning	1	PASS	4/15 FAIL	39/39 PASS	100.0%	0
RegFile	Design Planning	2	PASS	14/14 PASS	39/39 PASS	100.0%	0
RegFile	Direct	1	PASS	4/15 FAIL	39/39 PASS	100.0%	0
RegFile	Direct	2	PASS	14/14 PASS	39/39 PASS	100.0%	0
RegFile	Rules & Constraints	1	PASS	4/15 FAIL	39/39 PASS	100.0%	0
RegFile	Rules & Constraints	2	PASS	14/14 PASS	39/39 PASS	100.0%	0

Table 2. Complete initial per-run results. “Repairs” is the total Phase C repair count for the run; repaired RTL versions are analyzed separately below.

4.1 Aggregate Prompt-Style Results

Prompt style	Runs	Mean instructor score	Instructor suite pass	Generated TB suite pass	Generated TB case pass
Direct	8	98.12%	6/8 (75.0%)	4/8 (50.0%)	129/152 (84.9%)
Rules & Constraints	8	100.00%	8/8 (100.0%)	4/8 (50.0%)	133/153 (86.9%)
Design Planning	8	98.12%	5/8 (62.5%)	4/8 (50.0%)	132/153 (86.3%)

Table 3. Prompt-style aggregates. The generated-TB binary pass rate is 50% for each style (4 of 8 suites), but the individual test-case pass rates differ: Direct 84.9%, Rules & Constraints 86.9%, and Design Planning 86.3%. Thus, 50% does not mean that only half of all generated tests passed.

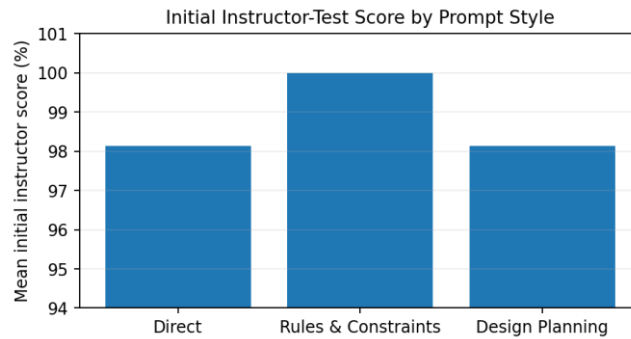


Figure 2. Mean initial instructor-test score by prompt style.

Rules & Constraints produced the highest observed mean instructor score at 100%. Direct and Design Planning both averaged 98.125%. Because there are only two runs per block/style cell, these results support an observed ranking for this experiment rather than a universal claim about prompt superiority.

5. Task 2 — Prompt-Style Analysis

5.1 Direct Prompt Style

The Direct style used the least additional instruction and still achieved a 98.125% mean instructor score. It produced several fully correct RTLs, including all six Extend/RegFile cases at the instructor level. Its Controller results were more variable: one initial run scored 37/40 while another had the same instructor score but a different generated-testbench outcome. The style demonstrates that a concise prompt can work well when the specification is sufficiently explicit, but it gives the model fewer reminders about subtle encoding and corner-case behavior.

5.2 Rules & Constraints Prompt Style

Rules & Constraints achieved the best aggregate initial instructor score: 100% across all eight runs. Its explicit constraints emphasize exact interfaces, widths, signedness, arithmetic, logic type, and Icarus compatibility. The benefit was most visible on ALU and Controller, while RegFile and Extend were already highly reliable under every style. The result suggests that explicit implementation constraints can reduce specification-translation errors without changing the underlying model.

5.3 Design Planning Prompt Style

Design Planning matched Direct at 98.125% mean instructor score. The planning-oriented prompt encourages the model to reason about interfaces and corner cases before implementation, but the experiment shows that additional planning instructions do not guarantee repair convergence. The ALU Design Planning run 1 remained at 36/40 through two repairs and failed compilation after the third repair.

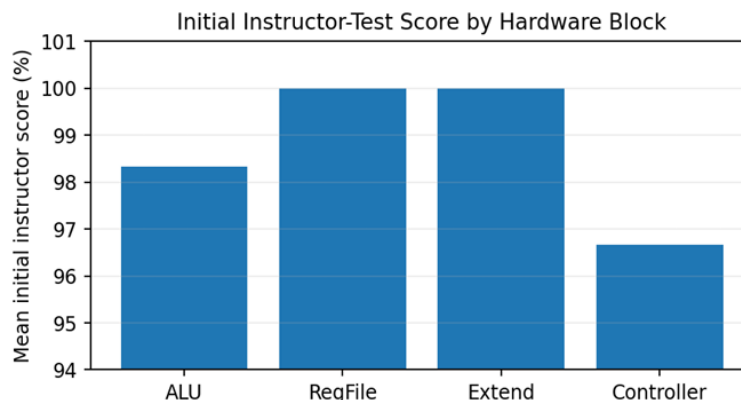


Figure 3. Mean initial instructor-test score by hardware block.

6. Iterative Repair and Failure Analysis

Repair accounting: Exactly **5** of 24 runs entered the repair loop, producing **7** repair calls. Four repaired Controller runs reached 40/40 after one repair. One ALU run used all three allowed repairs and did not converge. Extend and RegFile required no repair calls.

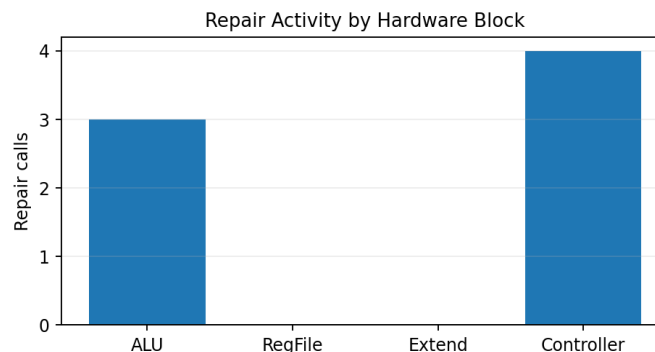


Figure 4. Repair activity by hardware block.

6.1 Repair Outcomes

Block	Style	Run	Initial	Repair calls	Final instructor	Outcome
ALU	Design Planning	1	36/40	3	Compile FAIL	Not recovered
Controller	Design Planning	1	39/40	1	40/40	Recovered
Controller	Design Planning	2	39/40	1	40/40	Recovered
Controller	Direct	1	37/40	1	40/40	Recovered
Controller	Direct	2	37/40	1	40/40	Recovered

Table 4. Runs that entered Phase C repair. Repaired versions are separate from the initial RTL results in Task 1.

6.2 Failure Modes and Repair Behavior

The Controller failures were concentrated on illegal-instruction handling. The instructor vectors exposed cases that the generated testbenches did not consistently exercise. After the failure evidence was included in the repair prompt, the repaired Controller RTLs reached 40/40 in one step.

7. Task 3 — Selection of Final RTL

Task 3 selects only the final RTL block for each of the four hardware blocks. Generated-testbench performance is intentionally excluded from this ranking so that RTL quality is judged independently by the authoritative instructor suite.

7.1 RTL Selection Criteria

Primary: highest instructor-suite score. Secondary: fewer repair attempts. Tie-break: first run among equally correct candidates. Repaired RTL is eligible, but an unrepaired version is preferred when the score is equal.

Block	Selected RTL	Instructor result	Why selected
ALU	Direct run 1 / initial_rtl.sv	40/40	Direct prompt produced a compact always_comb datapath with explicit shamt handling, independent Sum, safe defaults, and clear case-based operation mapping.
RegFile	Design Planning run 1 / initial_rtl.sv	39/39	Design Planning produced clean negedge write/reset logic and separate combinational read ports, with explicit x0 protection and reset priority.
Extend	Design Planning run 1 / initial_rtl.sv	40/40	Design Planning produced a compact case mapper with direct bit slices for all five formats and an explicit default, making the mapping easy to audit.
Controller	Design Planning run 1 / initial_rtl.sv	40/40	Design Planning produced a structured decoder with safe defaults and funct3/funct7 legality checks, including explicit IllegalInstrD handling.

8. Task 4 — LLM-Generated Self-Checking Testbenches

Task 4 evaluates the generated testbench as a verification artifact, independently from RTL selection. A testbench is preferred when its own generated suite completely passes, followed by the largest number of passed checks among passing suites; the first qualifying run is the tie-break. This prevents a strong RTL from causing a weak testbench to be selected.

8.1 Testbench Selection and Coverage

Selection criteria: complete binary PASS of the generated suite; then the largest number of passed checks among passing suites; then the first qualifying run. The selected testbench is therefore chosen independently of the selected RTL.

Block	Selected TB / result	What the selected testbench checks	Why selected
ALU	Direct run 1 / alu_tb.sv 18/18 PASS	ADD/SUB, logical operations, signed and unsigned comparisons, left/right shifts, signed arithmetic shift, wraparound, and shift-amount handling. It compares both ALUResult and Sum.	Complete passing suite; 18/18 checks.
RegFile	Direct run 2 / regfile_tb.sv 14/14 PASS	x0 staying zero, writes and overwrites, reset behavior and priority, dual combinational read ports, disabled writes, and boundary registers x1/x31.	Complete passing suite; 14/14 checks.
Extend	Direct run 2 / extend_tb.sv 22/22 PASS	I-, S-, B-, J-, and U-type immediate construction, sign extension, fragmented immediate fields, and representative positive/negative boundary values.	Complete passing suite; 22/22 checks.
Controller	Design Planning / controller_tb.sv 21/21 PASS	Legal R/load/store/branch/jump encodings and illegal encodings, while comparing the full decoded control bundle including RegWriteD, immediate source, ALU controls, memory controls, branch/jump controls, and IllegalInstrD.	Complete passing suite; 21/21 checks.

Overall generated-testbench context: 12/24 suites passed completely (50.0%), while 394/458 individual checks passed (86.0%). These are diagnostic results; the selected TBs above are complete passing verification artifacts.

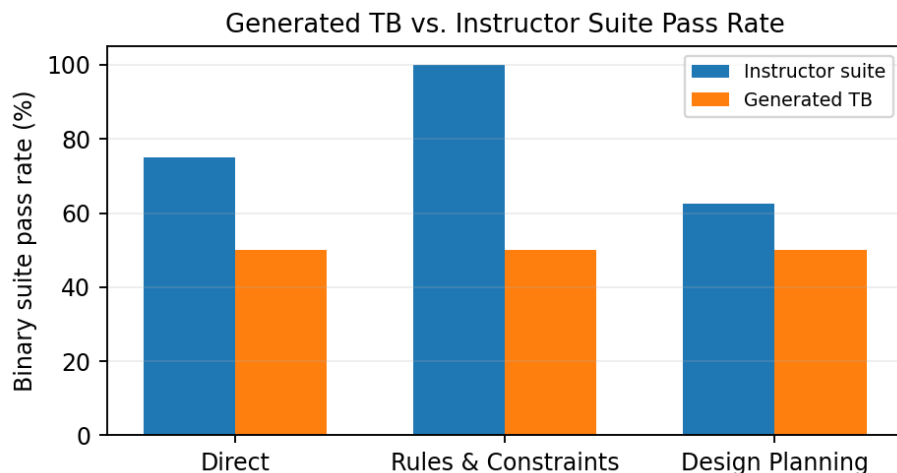


Figure 5. Binary suite pass rates by prompt style. Each style happened to have 4/8 generated suites pass, while individual test-case rates differ.

The block-level behavior is more informative than the binary style average. Extend passed every generated suite. RegFile had a strong run-to-run split: each style's run 1 generated 4/15 checks successfully, while each run 2 generated 14/14. ALU generated suites ranged from 15/19 to 20/20, and Controller ranged from 15/20 to 22/22. These differences show that generated verification quality can vary substantially even when the RTL itself is correct under the instructor suite.

The generated testbench is therefore best treated as an additional verification artifact and debugging aid. It is not independent from the LLM that generated the RTL, and the Controller results demonstrate that it can miss exactly the illegal-instruction cases that distinguish an almost-correct RTL from a fully correct one.

9. LLM Token Usage and Agentic Cost

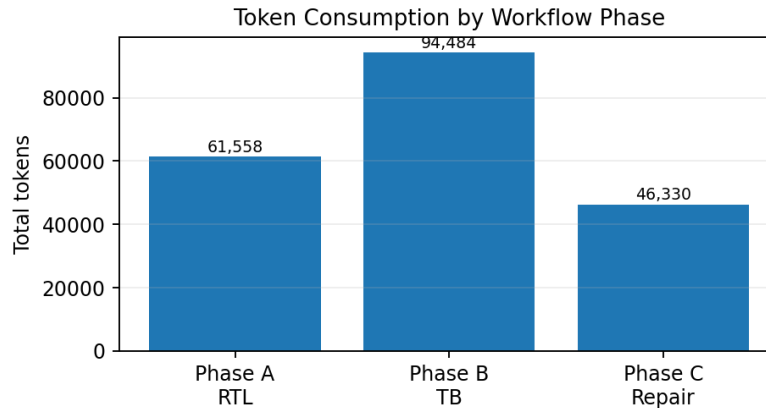


Figure 6. Total recorded token consumption by workflow phase.

Phase	Prompt tokens	Completion tokens	Total tokens	LLM calls
Phase A — RTL	32,672	28,886	61,558	24
Phase B — TB	45,984	48,500	94,484	24
Phase C — Repair	35,256	11,074	46,330	7
Total	113,912	88,460	202,372	55

Table 6. Aggregate token accounting. Phase C contains only actual repair calls; attempt 0 is a tool-execution step and therefore has zero LLM tokens.

The 24 runs required 48 baseline calls—one RTL-generation call and one testbench-generation call per run—plus seven repair calls. Phase B consumed more tokens than Phase A because self-checking testbenches generally contain more implementation text and verification logic. The parser also preserves Phase C token counts separately for attempts 1, 2, and 3, enabling deeper analysis of repair cost.

10. Task 5 — Lessons Learned

10.1 Prompt structure helps, but does not dominate

Rules & Constraints was the strongest observed prompt style, but the gap over Direct and Design Planning was only 1.875 percentage points in mean instructor score. The experiment therefore supports using explicit constraints, while also showing that prompt style should not be treated as the sole determinant of hardware correctness.

10.2 Independent verification is indispensable

A self-checking testbench generated by the same model family can share blind spots with the RTL. Instructor tests exposed illegal-instruction behavior that some generated testbenches omitted. A robust agent should therefore combine model-generated verification with an independent oracle or independently generated verification source.

10.3 Repair needs diversity and convergence control

The Controller repairs demonstrate that concrete failing vectors can enable fast recovery. The ALU repair demonstrates that repeated edits can stagnate or regress. Future agents should vary implementation strategy after repeated identical failures, use coverage/failure clustering, and stop when changes cease to improve the authoritative score.

10.4 Output limits are a real system resource

The fixed 4096-token completion limit influenced testbench design. The compact verification prompt intentionally prioritized a complete compilable testbench over exhaustive repetition. This is a general agent-design lesson: generation limits should be considered when designing the prompt and verification strategy.

10.5 Logging enables reproducible research

Exact prompts, full responses, model/settings metadata, token usage, RTL versions, testbenches, and simulation logs transform an LLM experiment into an auditable engineering workflow. The normalized parser outputs make it possible to regenerate report tables and figures without rerunning the model.

11. Reproducibility and Submission Structure

The complete workflow runs are organized under results/<block>/<prompt_style>/runN/. The prompt selection switch is controlled by --prompt-style <direct|rules_constraints|design_planning>. The run directory contains the initial RTL, generated testbench, repaired RTL versions when applicable, exact prompts, full responses, model metadata, simulation logs, and final RTL.

11.1 Commands Used

The following commands were used for the 24 experimental runs. The same six-command pattern was repeated for RegFile, Extend, and Controller with their corresponding specification paths and output directories.

```
mkdir -p
results/{alu,regfile,extend,controller}/{direct,rules_constraints,design_planning}/{run1,run2
}

python student_code/lab1.py --spec specs/alu.yaml --out-dir results/alu/direct/run1 --
provider tamu --prompt-style direct 2>&1 | tee results/alu/direct/run1/terminal.log
python student_code/lab1.py --spec specs/alu.yaml --out-dir results/alu/direct/run2 --
provider tamu --prompt-style direct 2>&1 | tee results/alu/direct/run2/terminal.log

python student_code/lab1.py --spec specs/alu.yaml --out-dir
results/alu/rules_constraints/run1 --provider tamu --prompt-style rules_constraints 2>&1 |
tee results/alu/rules_constraints/run1/terminal.log
python student_code/lab1.py --spec specs/alu.yaml --out-dir
results/alu/rules_constraints/run2 --provider tamu --prompt-style rules_constraints 2>&1 |
tee results/alu/rules_constraints/run2/terminal.log

python student_code/lab1.py --spec specs/alu.yaml --out-dir results/alu/design_planning/run1
--provider tamu --prompt-style design_planning 2>&1 | tee
results/alu/design_planning/run1/terminal.log
python student_code/lab1.py --spec specs/alu.yaml --out-dir results/alu/design_planning/run2
--provider tamu --prompt-style design_planning 2>&1 | tee
results/alu/design_planning/run2/terminal.log
```

For terminal capture, the output directory was created first and stdout/stderr were captured with tee. The full prompt-style execution variants, instructor sample-test commands, generated-testbench commands, environment setup, and submission organization are documented in **README.txt**.

The README should be included with the submission so that the exact environment and command sequence remains available without reproducing a long command appendix in the report.

12. Conclusion

The experiment demonstrates a working agentic RTL-design pipeline in which an LLM generates hardware and verification artifacts, conventional tools execute those artifacts, and concrete test evidence drives bounded repair. Across 24 runs, initial RTL quality was high, with a 98.75% mean instructor score, but the experiment also exposed failure modes that would be difficult to detect using only model-generated verification.

Rules & Constraints produced the best observed initial mean instructor score, while Controller repair runs showed that targeted external evidence can enable rapid correction. At the same time, the ALU non-convergent repair shows that LLM repair is not inherently monotonic. The generated testbench results further show why an independent correctness oracle is essential.

Final engineering conclusion: *The most trustworthy LLM hardware workflow is not “prompt → RTL.” It is “specification → constrained generation → independent verification → evidence-driven bounded repair → complete artifact logging.” The experiment supports this architecture and provides a reproducible basis for future improvements in repair diversity, coverage, and verification independence.*

Appendix A — Exact Prompt Templates

A.1 Direct

Generate the SystemVerilog RTL for the following hardware specification.

Top module: {top_module}

Hardware specification:
{content}

Implement the specification exactly. Preserve the specified module interface and all defined behavior. Return the complete RTL implementation in a single fenced code block.

A.2 Rules & Constraints

Generate the SystemVerilog RTL for the following hardware specification.

Top module: {top_module}

Hardware specification:
{content}

Implement the specification exactly and follow these requirements: (1) preserve exact module name, port names, directions, and widths; (2) implement all operations and control conditions; (3) pay careful attention to widths, signedness, arithmetic, comparisons, and boundary conditions; (4) use combinational or sequential logic as specified; (5) do not add unspecified behavior; (6) produce synthesizable SystemVerilog compatible with Icarus Verilog.

Return the complete implementation in a single fenced code block.

A.3 Design Planning

Design the following hardware block in SystemVerilog.

Top module: {top_module}

Hardware specification:
{content}

Before writing code, work through the design step by step: identify inputs/outputs and widths; identify operations and control conditions; identify important corner cases including reset, boundary values, overflow, and specified cases; determine combinational versus sequential logic; and use this analysis to determine the RTL structure. Then implement the design.

Preserve the exact interface and all defined behavior. Return the complete SystemVerilog implementation in a single fenced code block.

A.4 Fixed Phase A System Prompt

You are an RTL design agent responsible for translating hardware specifications into SystemVerilog. Use the hardware specification provided by the user as the source of truth for the requested design. Your goal is to produce a correct implementation that follows the provided specification.

Appendix B — Per-Run Token and Outcome Record

The machine-readable `run_summary.csv` contains individual prompt/completion/total token counts for Phase A, Phase B, and each Phase C repair attempt. The following compact table gives the per-run phase totals and initial instructor score.

Block	Style	Run	A total	B total	C total	All tokens	Repairs	Initial instructor
ALU	Design Planning	1	2,153	3,516	10,563	16,232	3	90.0%
ALU	Design Planning	2	2,526	3,478	0	6,004	0	100.0%
ALU	Direct	1	1,853	3,522	0	5,375	0	100.0%
ALU	Direct	2	1,879	3,516	0	5,395	0	100.0%
ALU	Rules & Constraints	1	1,892	3,516	0	5,408	0	100.0%
ALU	Rules & Constraints	2	1,907	3,478	0	5,385	0	100.0%
Controller	Design Planning	1	5,598	5,771	9,109	20,478	1	97.5%
Controller	Design Planning	2	5,873	6,141	9,724	21,738	1	97.5%
Controller	Direct	1	4,321	5,771	8,201	18,293	1	92.5%
Controller	Direct	2	4,582	6,141	8,733	19,456	1	92.5%
Controller	Rules & Constraints	1	5,481	5,771	0	11,252	0	100.0%
Controller	Rules & Constraints	2	5,596	6,141	0	11,737	0	100.0%
Extend	Design Planning	1	1,366	2,775	0	4,141	0	100.0%
Extend	Design Planning	2	1,731	2,916	0	4,647	0	100.0%
Extend	Direct	1	1,181	2,775	0	3,956	0	100.0%
Extend	Direct	2	1,180	2,916	0	4,096	0	100.0%
Extend	Rules & Constraints	1	1,276	2,775	0	4,051	0	100.0%
Extend	Rules & Constraints	2	1,278	2,916	0	4,194	0	100.0%
RegFile	Design Planning	1	1,697	3,369	0	5,066	0	100.0%
RegFile	Design Planning	2	2,011	3,514	0	5,525	0	100.0%
RegFile	Direct	1	1,505	3,369	0	4,874	0	100.0%
RegFile	Direct	2	1,514	3,514	0	5,028	0	100.0%
RegFile	Rules & Constraints	1	1,581	3,369	0	4,950	0	100.0%
RegFile	Rules & Constraints	2	1,577	3,514	0	5,091	0	100.0%

Table 7. Per-run token totals and initial instructor scores. See `run_summary.csv` for separate prompt/completion counts for repair attempts 1–3.

Appendix C — Artifact Checklist

- **initial_rtl.sv** — first LLM-generated RTL.
- **<module>_tb.sv** — LLM-generated self-checking testbench.
- **revised_rtl_N.sv** — each repair candidate, when repair occurs.
- **final_rtl.sv** — final RTL selected by the workflow.
- **phaseA_prompt_*.txt / phaseB_prompt_*.txt / phaseC_prompt_*.txt** — exact prompts.
- **phaseA_response.txt / phaseB_response.txt / phaseC_response_*.txt** — full model responses.
- **phaseA_metadata.txt / phaseB_metadata.txt / phaseC_metadata_*.txt** — model, settings, and token metadata.
- **phaseC_attempt_N.log and run.log** — simulation and decision history.
- **simulation_build/** — generated simulation executables/artifacts.
- **terminal.log** — captured terminal execution output.
- The parser outputs `run_summary.csv`, `attempts.csv`, `llm_calls.csv`, `failures.csv`, and `report_data.json` provide a normalized analysis layer for the report.

This appendix is intentionally concise because the complete artifact hierarchy and exact execution procedure are documented in `README.txt`. The original run directories remain the provenance sources for every reported number.